

CST-407 Milestone 1 Implement Spring Boot Security

Alex M. Frear

College of Science, Engineering, and Technology, Grand Canyon University

Course Number: CST-407

Professor: Dr. Melody White

08/22/2025

Link to Screencast: [CST407 - Milestone 1 - Events App](#)

Table of Contents

Section 1 — Environment & How to Run	4
1.1 Prerequisites	4
1.2 Database Setup	4
1.3 Application Configuration	4
1.4 Build & Run	4
1.5 First-Time Usage	5
1.6 Troubleshooting	5
Section 2 — Features Implemented	6
2.1 Authentication & Registration	6
2.2 Roles & Authorization	6
Route Matrix	6
2.3 Ownership Model	6
2.4 UI/Template Layer (Thymeleaf)	7
2.5 Persistence (JPA)	7
2.6 Security Configuration (summary)	7
Section 3 — Evidence & Screenshots	8
Part A — Public vs. Auth Flow	8
Logged-out Events List — Route: /events	8
Register — Route: /users/register	9
Login — Route: /users/loginForm	10
Part B — Role-based Visibility & Ownership	11
Owner View (normal user) — Route: /events as alex	11
Non-owner (another normal user) — Route: /events as testuser	12
Admin View — Route: /events as admin	13
Part C — Database Proof	14
Users Table (BCrypt)	14
Roles Table (RBAC)	15
Part D — Runtime Evidence	16
App Startup / DB Connection	16
Section 4 — Roles & Admin Promotion (SQL)	17
4.1 Default Role Behavior	17
4.2 Promote a User to Admin	17
4.3 Verify Roles	17
4.4 Remove Admin (if needed)	17
4.5 Notes	18
Section 5 — Test Cases (Step-by-step)	19

Functional & RBAC Tests	19
T1 — Register	19
T2 — Login / Logout	19
T3 — Create Event (auth required)	19
T4 — Edit Own Event	19
T5 — Delete Own Event	19
T6 — Non-owner cannot edit.....	20
T7 — Admin can edit/delete any event	20
T8 — Auth redirect when logged out.....	20
T9 — Search	20
UI Visibility Tests	20
U1 — Navbar states	20
U2 — Actions column.....	20
Database Tests	21
D1 — Users table.....	21
D2 — Roles table.....	21
<i>Section 6 — Conclusion</i>	<i>22</i>

Section 1 — Environment & How to Run 1.1 Prerequisites

- **JDK:** Java 17
- **Build:** Maven Wrapper (`mvnw`, `mvnw.cmd`)
- **Database:** MySQL (MAMP) on **127.0.0.1:8889**
- **Optional tools:** MySQL Workbench for inspecting tables

1.2 Database Setup

1. Start MySQL via **MAMP** (confirm port **8889**).
2. Create the schema (once):

```
CREATE DATABASE IF NOT EXISTS eventsapp;
```

3. (Optional) Verify the `eventsapp` schema exists in MySQL Workbench.

1.3 Application Configuration

Edit `src/main/resources/application.properties` (use placeholders below):

```
spring.datasource.url=jdbc:mysql://127.0.0.1:8889/eventsapp?useSSL=false&allowPublicKeyRetrieval=true&serverTimezone=UTC
spring.datasource.username=<DB_USER>          # e.g., root
spring.datasource.password=<DB_PASSWORD>      # e.g., root
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
```

1.4 Build & Run

From the project root:

```
./mvnw clean spring-boot:run
```

Then open: <http://localhost:8080>

1.5 First-Time Usage

1. Visit **/users/register** to create a user.
2. Log in at **/users/loginForm**.
3. Create an event at **/events/create**.

(Grant admin later via a one-line SQL in the Roles section.)

1.6 Troubleshooting

- **DB connection errors:** Ensure MAMP MySQL is running on **127.0.0.1:8889** and credentials match `application.properties`.
- **Port 8080 in use:** Stop the other app or run with `--server.port=8081`.
- **Tables missing:** Confirm schema name `eventsapp` and keep `spring.jpa.hibernate.ddl-auto=update`.

Section 2 — Features Implemented

2.1 Authentication & Registration

- **User registration** with server-side validation.
- **BCrypt password hashing** on save (hashes begin with \$2... in the DB).
- **Custom login page** (/users/loginForm) wired to Spring Security (POST /login, fields userName & password).
- **Logout** via POST /logout, with navbar button styled as a link.

2.2 Roles & Authorization

- Roles persisted in DB (roles table):
 - ROLE_USER (assigned by default to new users)
 - ROLE_ADMIN (granted via a single SQL statement)
- **Access rules (server-enforced):**
 - Anyone (no login): view list /events, search /events/search
 - Authenticated users: create /events/create
 - **Owner**: edit/delete **their own** events
 - **Admin**: edit/delete **any** event

Route Matrix

Route/Action	Anonymous	Authenticated	Owner	Admin
View events /events	✓	✓	✓	✓
Search /events/search	✓	✓	✓	✓
Create /events/create	✗	✓	✓	✓
Edit /events/edit/{id}	✗	✗*	✓	✓
Delete /events/delete/{id}	✗	✗*	✓	✓

* Non-owner, non-admin users are blocked by the controller even if they guess a URL.

2.3 Ownership Model

- On **create**, the controller sets organizerid to the **logged-in user** (owner).
- On **edit/delete**, the controller checks:
 - If **owner** → allowed
 - Else if **admin** → allowed
 - Else → redirected with a friendly message
- UI mirrors these rules so buttons only appear when appropriate (owner or admin).

2.4 UI/Template Layer (Thymeleaf)

- **Layout + Navbar** (`layout.html`) with Spring Security dialect:
 - Shows **Register/Login** when logged out.
 - Shows **Create Event, Hello, {username}, and Logout** when logged in.
 - (Optional) **Admin** link appears only for `ROLE_ADMIN`.
- **Events list** (`events.html`):
 - “Create New Event” link only when logged in.
 - **Actions** (Edit/Delete) visible:
 - to **admins** for all events,
 - to **owners** for their own events only.
- **Login/Register** pages use the shared layout so the navbar is consistent.

2.5 Persistence (JPA)

- Entities backed by MySQL (MAMP).
- JPA/Hibernate with `ddl-auto=update` creates/updates tables automatically.
- Repositories/services handle CRUD for `users`, `roles`, and `events`.

2.6 Security Configuration (summary)

- Spring Security `SecurityFilterChain`:
 - Permits `/`, `/events`, `/events/search`, `/users/*`, and static assets.
 - Requires authentication for `/events/create`, `/events/edit/**`, `/events/delete/**`.
 - Custom login page at `/users/loginForm`, login processing at `/login`.
 - Logout at `/logout` (POST).
- Password encoding via `BCryptPasswordEncoder`.
- `UserDetailsService` bridges app users/roles to Spring Security.

Note: For milestone simplicity, CSRF may be disabled in config; switching Delete to **POST** with CSRF enabled is a recommended hardening step for later.

Section 3 — Evidence & Screenshots

Part A — Public vs. Auth Flow

Logged-out Events List — *Route: /events*

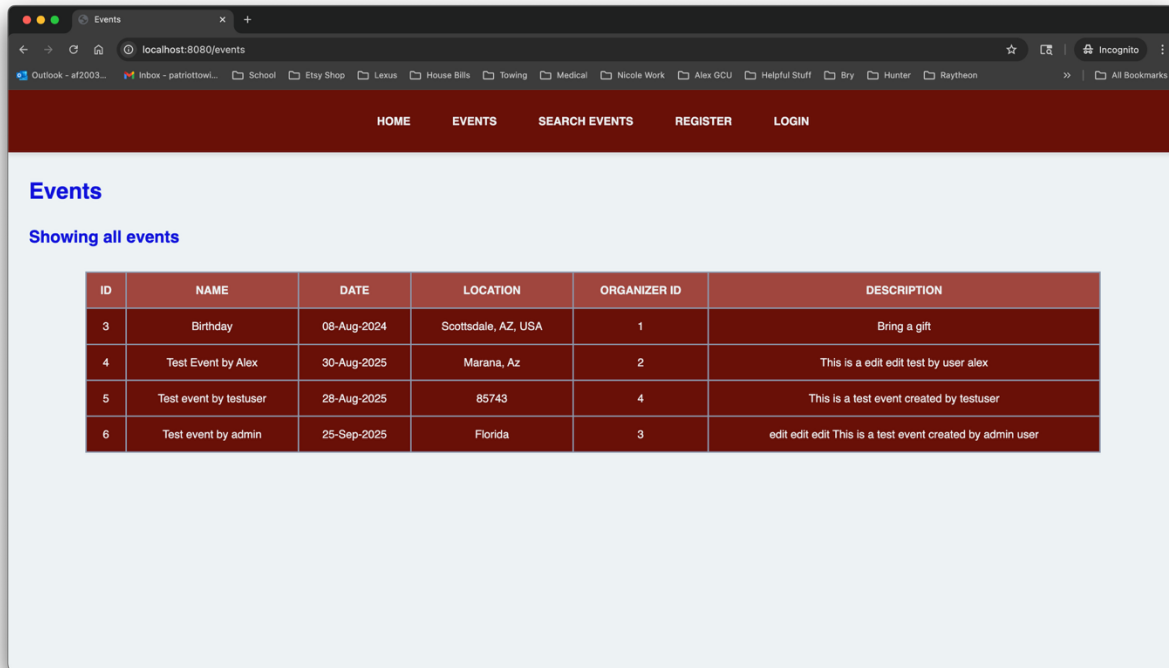


Figure 1 Public view: no 'Create New Event', no 'Actions' column. Navbar shows Register/Login.

Register — *Route: /users/register*

The screenshot shows a web browser window with the address bar displaying `localhost:8080/users/register`. The browser's tab is labeled "Events App". The page features a dark red navigation bar with the following links: HOME, EVENTS, SEARCH EVENTS, REGISTER, and LOGIN. The main content area is light blue and contains the heading "Register" in bold blue text. Below the heading are two input fields: "Username" and "Password". Under the "Password" field is a dark red button labeled "CREATE ACCOUNT" and a blue link labeled "Back to Login".

Figure 2 Registration form with Username & Password fields.

Login — *Route:* /users/loginForm

The screenshot shows a web browser window with the address bar displaying `localhost:8080/users/loginForm?registered`. The browser's bookmark bar shows various folders like 'Outlook', 'Inbox', 'School', 'Etsy Shop', 'Lexus', 'House Bills', 'Towing', 'Medical', 'Nicole Work', 'Alex GCU', 'Helpful Stuff', 'Bry', 'Hunter', and 'Raytheon'. The page features a dark red navigation bar with the following links: [HOME](#), [EVENTS](#), [SEARCH EVENTS](#), [REGISTER](#), and [LOGIN](#). The main content area is light blue and contains a 'Login' section. It includes a message 'Account created. Please sign in.', input fields for 'Username' and 'Password', a 'SIGN IN' button, and a link 'Need an account? Register'.

Figure 3 Custom login page (shows redirect banner when coming from register).

Part B — Role-based Visibility & Ownership

Owner View (normal user) — *Route: /events* as **alex**

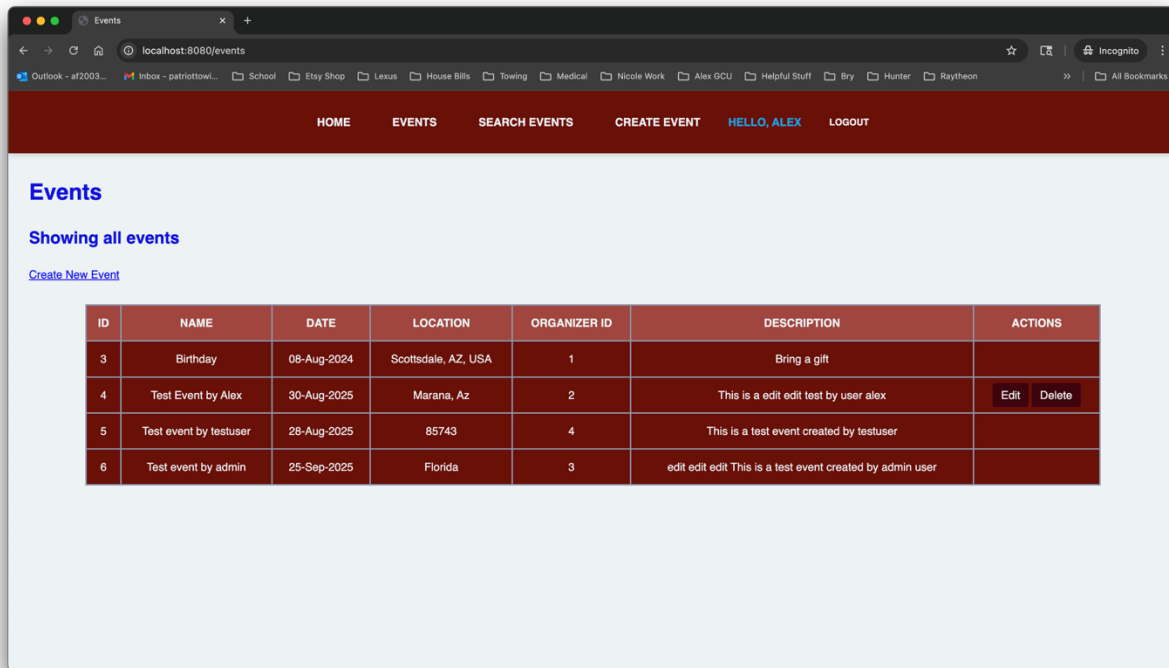


Figure 4 Edit/Delete visible *only* for alex's own event.

Non-owner (another normal user) — *Route: /events* as testuser

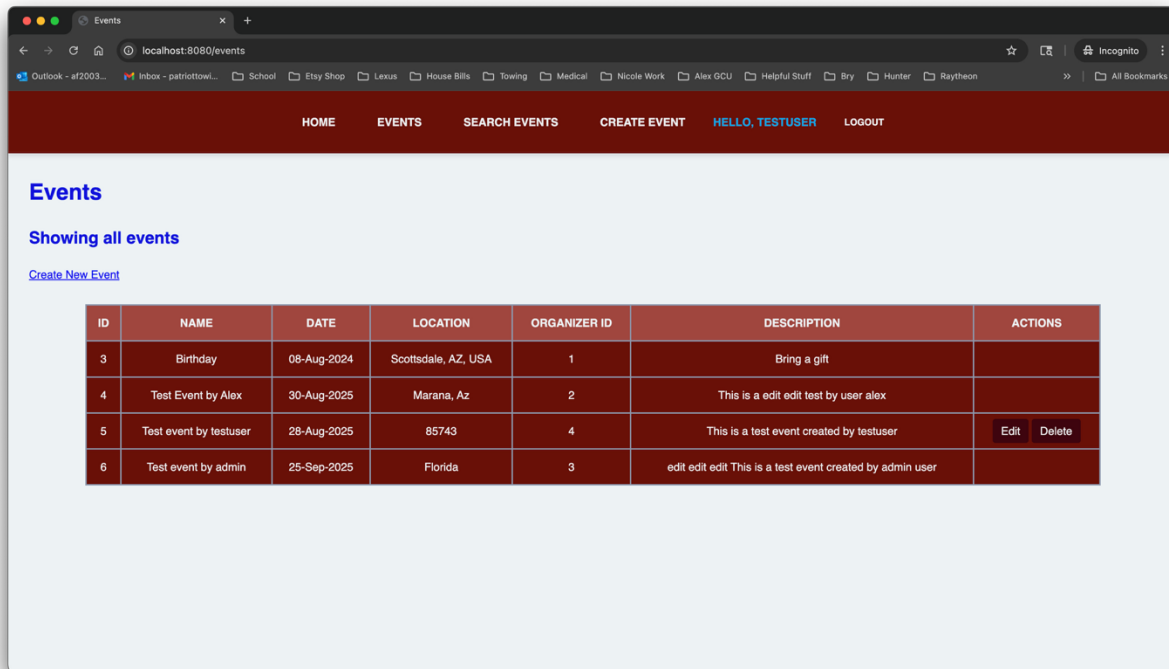


Figure 5 No Edit/Delete on alex's events; actions only on testuser's own event.

Admin View — *Route: /events* as admin

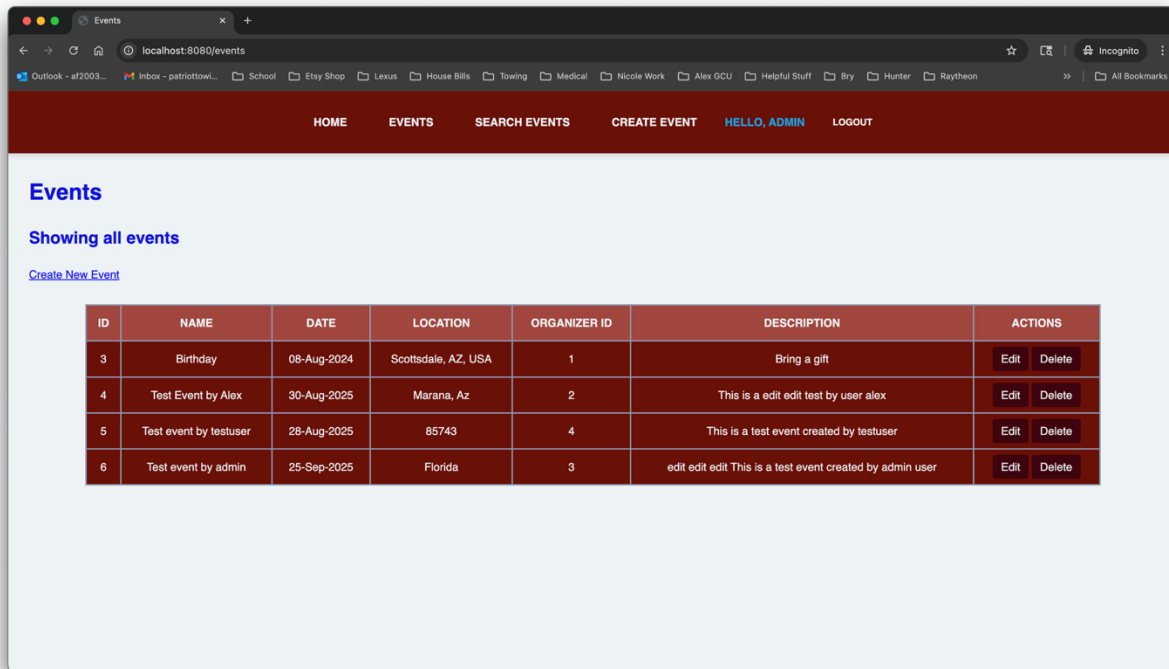
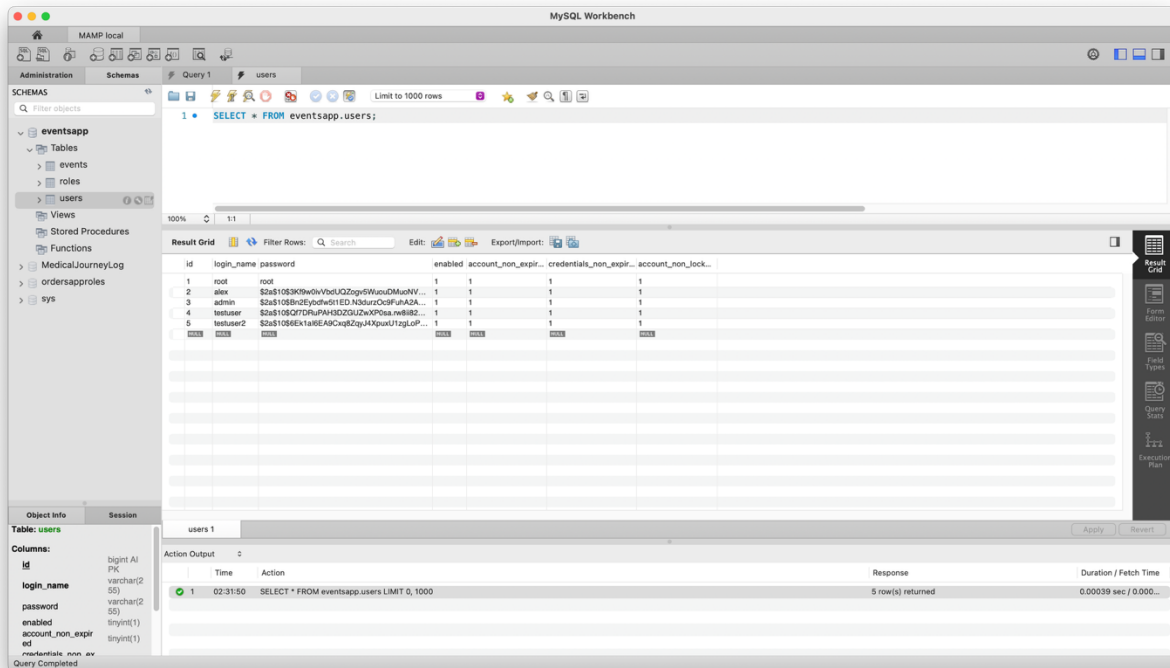


Figure 6 Admin sees Edit/Delete on *all* events.

Part C — Database Proof

Users Table (BCrypt)



MySQL Workbench

Administration Schemas Query 1 users Limit to 1000 rows

1 • SELECT * FROM eventsapp.users;

Result Grid

	id	login_name	password	enabled	account_non_expir...	credentials_non_expir...	account_non_lock...
1	root	root		1	1	1	1
2	alex		\$2u\$10\$3K9w0vVb0UQZgpr9WouDMu0NV...	1	1	1	1
3	admin		\$2u\$10\$Bn05j0p0d0rEDN3kuz0d0FJHADA...	1	1	1	1
4	testuser		\$2u\$10\$Q7D7RufAH30Z0UzWXP0sa.w8l82...	1	1	1	1
5	testuser2		\$2u\$10\$6E1a8EAGCngZqJ4XpuU1zgLoP...	1	1	1	1

Object Info Session

Table: users

Columns:

- id: bigint AI PK
- login_name: varchar(255)
- password: varchar(255)
- enabled: tinyint(1)
- account_non_expir: tinyint(1)
- credentials_non_expir: tinyint(1)
- account_non_lock: tinyint(1)

Action Output

	Time	Action	Response	Duration / Fetch Time
1	02:31:50	SELECT * FROM eventsapp.users LIMIT 0, 1000	5 row(s) returned	0.00039 sec / 0.000...

Query Completed

Figure 7 eventsapp.users — passwords hashed (start with \$2...).

Roles Table (RBAC)

MySQL Workbench

Administration Schemas Query 1 users roles

Schemas

Filter objects

eventsapp

Tables

events

roles

users

Views

Stored Procedures

Functions

MedicalJourneyLog

ordersapproles

sys

1 • SELECT * FROM eventsapp.roles;

Limit to 1000 rows

100% 1:1

Result Grid

Filter Rows: Search Export

user_id	role
1	ROLE_USER
1	ROLE_ADMIN
2	ROLE_USER
3	ROLE_USER
3	ROLE_ADMIN
4	ROLE_USER
5	ROLE_USER

Object Info Session

Table: roles

Columns:

user_id bigint

role varchar(255)

roles 1

Action Output 5

	Time	Action	Response	Duration / Fetch Time
1	02:31:50	SELECT * FROM eventsapp.users LIMIT 0, 1000	5 row(s) returned	0.00039 sec / 0.000...
2	02:32:17	SELECT * FROM eventsapp.roles LIMIT 0, 1000	7 row(s) returned	0.00032 sec / 0.0000...

Query Completed

Figure 8 `eventsapp.roles` — all users have `ROLE_USER`; admin also has `ROLE_ADMIN`.

Part D — Runtime Evidence

App Startup / DB Connection

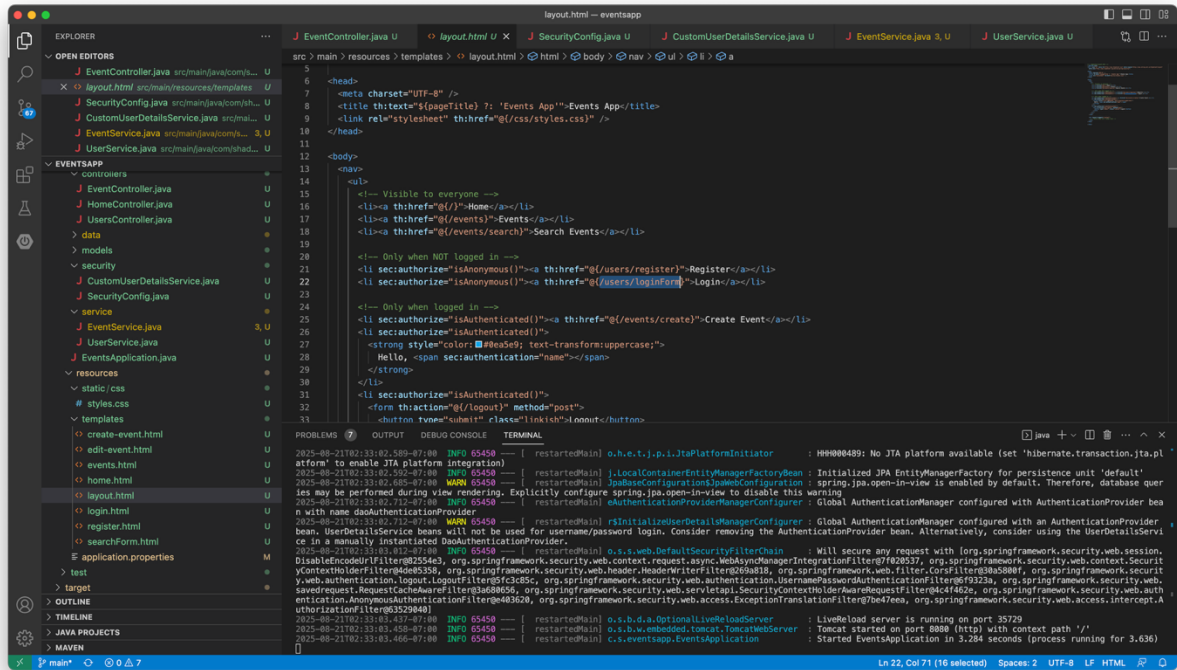


Figure 9 Spring Boot started on port 8080; Hikari/Tomcat logs show successful DB connection.

Section 4 — Roles & Admin Promotion (SQL)

4.1 Default Role Behavior

- New users registered via `/users/register` are saved with **ROLE_USER**.
- Roles are stored in table **eventsapp.roles** with columns: `user_id` (FK → `users.id`) and `role` (e.g., `ROLE_USER`, `ROLE_ADMIN`).

4.2 Promote a User to Admin

Use this one-time SQL to grant **ROLE_ADMIN** to the user named `admin` (or change the username as needed). The `NOT EXISTS` guard prevents duplicates.

```
INSERT INTO eventsapp.roles (user_id, role)
SELECT u.id, 'ROLE_ADMIN'
FROM eventsapp.users u
WHERE u.login_name = 'admin'
  AND NOT EXISTS (
    SELECT 1 FROM eventsapp.roles r
    WHERE r.user_id = u.id AND r.role = 'ROLE_ADMIN'
  );
```

Run this in **MySQL Workbench** connected to `eventsapp`.
After running it, log out and back in as `admin`.

4.3 Verify Roles

Run this query to confirm role assignments:

```
SELECT r.user_id, u.login_name, r.role
FROM eventsapp.roles r
JOIN eventsapp.users u ON u.id = r.user_id
ORDER BY r.user_id, r.role;
```

Expected: all users have `ROLE_USER`; the admin account also has `ROLE_ADMIN`.

4.4 Remove Admin (if needed)

To demote the admin account back to normal user:

```
DELETE r FROM eventsapp.roles r
JOIN eventsapp.users u ON u.id = r.user_id
WHERE u.login_name = 'admin' AND r.role = 'ROLE_ADMIN';
```

4.5 Notes

- The application enforces authorization on the server:
 - **Owner** can edit/delete their own events.
 - **Admin** can edit/delete any event.
- UI visibility (buttons/links) matches the server rules to avoid confusion.

Section 5 — Test Cases (Step-by-step)

Test data used:

alex (ROLE_USER), testuser (ROLE_USER), admin (ROLE_ADMIN).

Each user has at least one event.

Functional & RBAC Tests

T1 — Register

Steps: Open `/users/register`, submit username & password.

Expected: Redirects to `/users/loginForm?registered`; user created with ROLE_USER.

Result:  Pass

T2 — Login / Logout

Steps: Log in at `/users/loginForm`; observe navbar and events; log out.

Expected: Logged-in navbar shows Create Event / Hello {user} / Logout; logged-out shows Register / Login.

Result:  Pass

T3 — Create Event (auth required)

Steps: Log in as alex; open `/events/create`; create event; view `/events`.

Expected: New event visible; Actions show Edit/Delete on owner's row.

Result:  Pass

T4 — Edit Own Event

Steps: Logged in as alex, click **Edit** on alex's event; save changes.

Expected: Updated values shown in list.

Result:  Pass

T5 — Delete Own Event

Steps: Logged in as alex, click **Delete** on alex's event.

Expected: Row removed from list and database.

Result:  Pass

T6 — Non-owner cannot edit

Steps: Log in as `testuser`; manually open `/events/edit/{alexEventId}`.

Expected: Redirect back to `/events` with a “not your event” message.

Result:  **Pass**

T7 — Admin can edit/delete any event

Steps: Log in as `admin`; open `/events`; edit/delete other users’ events.

Expected: Admin can manage all rows.

Result:  **Pass**

T8 — Auth redirect when logged out

Steps: Logged out; open `/events/create`.

Expected: Redirect to `/users/loginForm`.

Result:  **Pass**

T9 — Search

Steps: Open `/events/search`; submit a keyword.

Expected: Results show only matching events; page displays the search term.

Result:  **Pass**

UI Visibility Tests

U1 — Navbar states

Expected:

- Logged out: Home, Events, Search Events, Register, Login
- Logged in (user/admin): Home, Events, Search Events, Create Event, Hello {user}, Logout

Result:  **Pass**

U2 — Actions column

Expected:

- Logged out: no Actions column
- User: actions only on owned rows
- Admin: actions on all rows

Result:  **Pass**

Database Tests

D1 — Users table

Query:

```
SELECT id, login_name, password, enabled
FROM eventsapp.users
ORDER BY id;
```

Expected: Passwords are BCrypt hashes (\$2...).

Result:  **Pass**

D2 — Roles table

Query:

```
SELECT r.user_id, u.login_name, r.role
FROM eventsapp.roles r
JOIN eventsapp.users u ON u.id = r.user_id
ORDER BY r.user_id, r.role;
```

Expected: All users have `ROLE_USER`; admin also has `ROLE_ADMIN`.

Result:  **Pass**

Section 6 — Conclusion

- Implemented a secure Spring Boot web app with **registration, login, role-based access**, and event **CRUD**.
- **Ownership enforcement**: only event owners can modify their events; **admins** can manage all.
- **Data integrity**: passwords stored with **BCrypt**; roles persisted in a separate table.
- UI uses Thymeleaf with a shared layout; navbar adapts to **auth state** and **role**.
- Tested end-to-end (users, admin, DB evidence). All tests **passed**.